Practical-1

1. Aim:

Create different blinking patterns with LEDs by connecting to GPIO pins of Raspberry Pi.

2. Prior Concepts:

i) Python Programming - loops, time delay, GPIO library

ii) Digital Electronics - LED works on HIGH (1) = ON, LOW (0) = OFF

3. New Concepts:

i) Raspi 4: It has 40 GPIO pins. You can control LEDs, sensors, motors from Python code.

ii) Hardware Interface: LED anode (+ve) goes to GPIO via 330 ohm resistor, cathode (-ve) to GND. Resistor protects LED from burning.

4. Procedure:

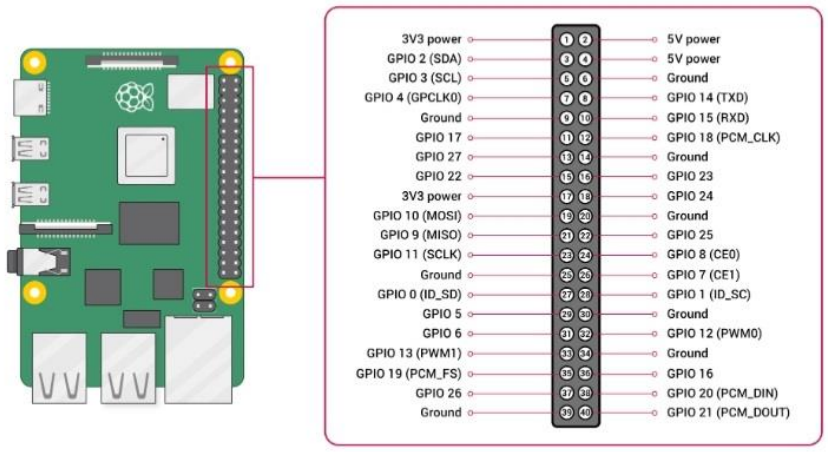
i) Install Python3 IDE:

Open terminal and type:

sudo apt install idle3

ii) Connection as per your diagram:

- Connect LEDs on breadboard
- Connect 330 ohm resistor to anode (long leg +ve) of LED
- Other end of resistor to GPIO pin (e.g., GPIO 17, 27, 22)
- Cathode (short leg -ve) to GND pin of Pi
- In your diagram: Right side is 40-pin header - connect as shown



5. Implementation with Python Code:

Question 1 : Write a program to generate TWO Bit binary Pattern.

Code :

```
import RPi.GPIO as GPIO
from time import sleep

GPIO.setmode(GPIO.BOARD)

led1=29  #Yellow
led2=31  #Green

GPIO.setup(led1,GPIO.OUT)
GPIO.setup(led2,GPIO.OUT)

GPIO.output(led1,False)
GPIO.output(led2,False)

def ledpattern(v1,v2):
    GPIO.output(led1,v1)
    GPIO.output(led2,v2)

try:
    while True:
        ledpattern(1,0)
        sleep(0.5)
        ledpattern(0,1)
        sleep(0.5)
        ledpattern(1,1)
        sleep(0.5)
        ledpattern(0,0)
        sleep(0.5)
```

finally:

GPIO.output(led1, False)

GPIO.output(led2, False)

GPIO.cleanup()

6. Results:

PRACTICAL -2

1.Aim:

Use Telegram to control Raspberry Pi GPIO.

2. Prior Concepts:

i) Python Programming

ii) MQTT Protocol

3. New Concepts:

Controlling LED through telegram bot and Raspi

4. Hardware Required:

i)LED

ii) Breadboard

iii) Resistor

iv) Jumper wires

5. Software Required:

On Mobile Phone: Telegram:

Steps:

Download Telegram from playstore on your android phone.

i) Install Telegram.

ii) Open Telegram app in your system or mobile

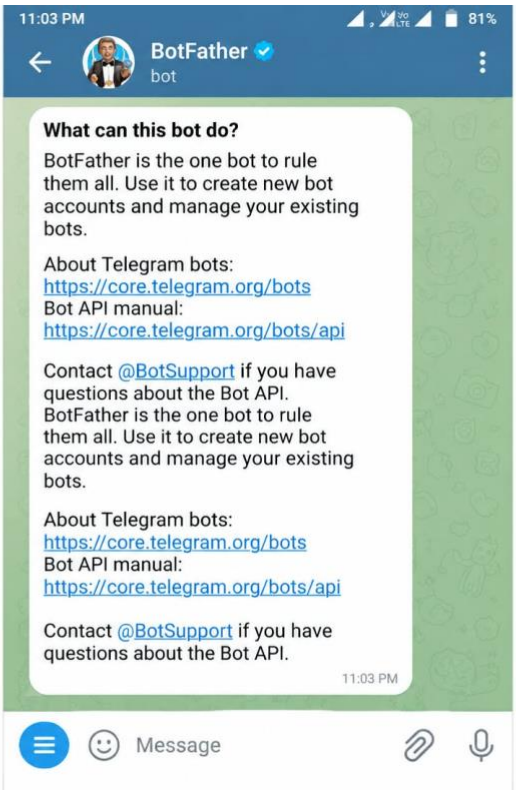
iii) Click On Start Messaging Button

iv) Enter your mobile number to register with telegram service.

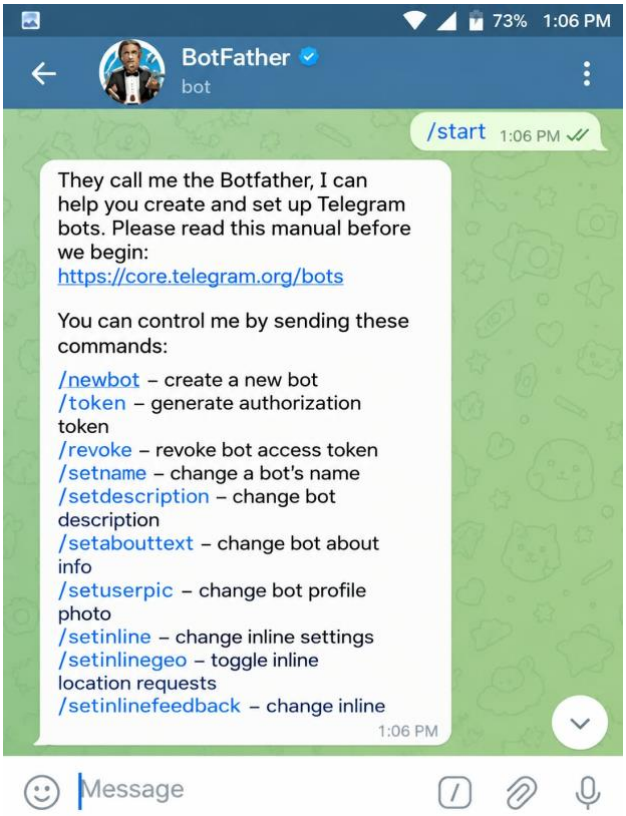
v)Search for name “BotFather”.



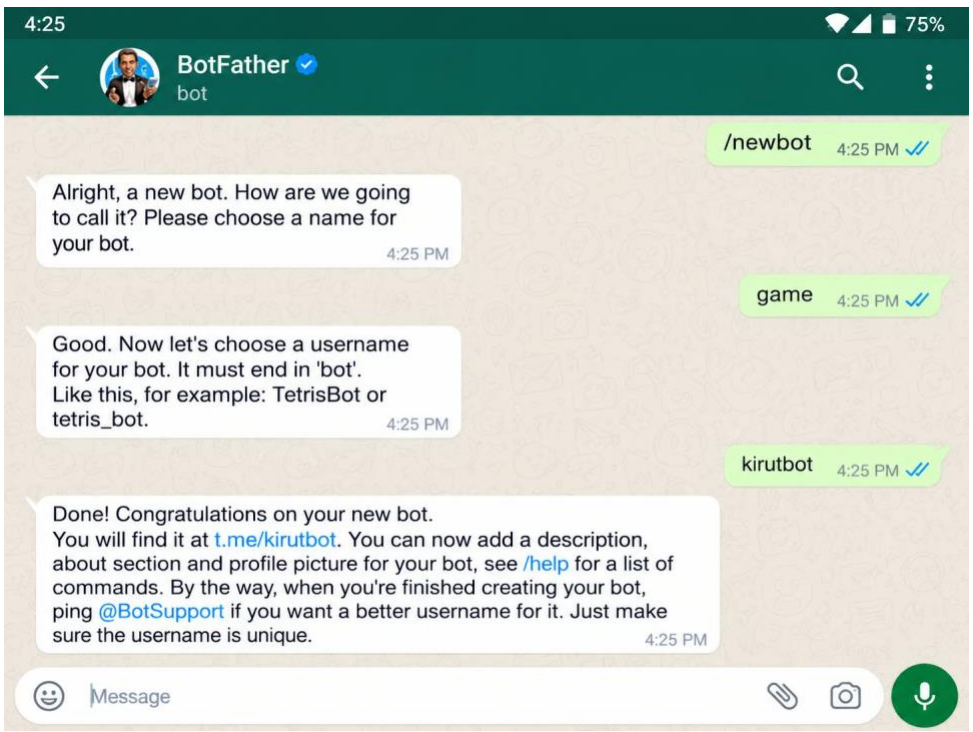
vi) Click on ”BotFather”



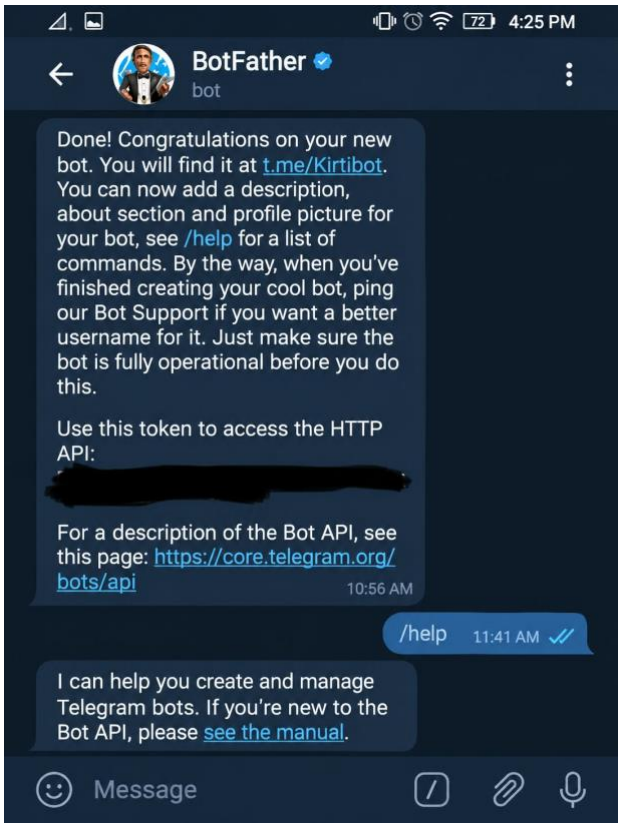
vii)To start “BotFather” type /start in message.



viii) Now type /newbot in message and then give name to your BOT and Username also:



ix) Obtain access token:



This same token we are supposed to use in our code in raspberry pi to connect to our BOT.

6.Procedure

Set up On Raspberry Pi:

1. Install "Python Package Index" and Telepot using:

python3 -m pip config set global.break-system-packages true

sudo apt install python3-pip

pip3 install telepot

2.Test your bot using python3 IDLE and type:

import telepot

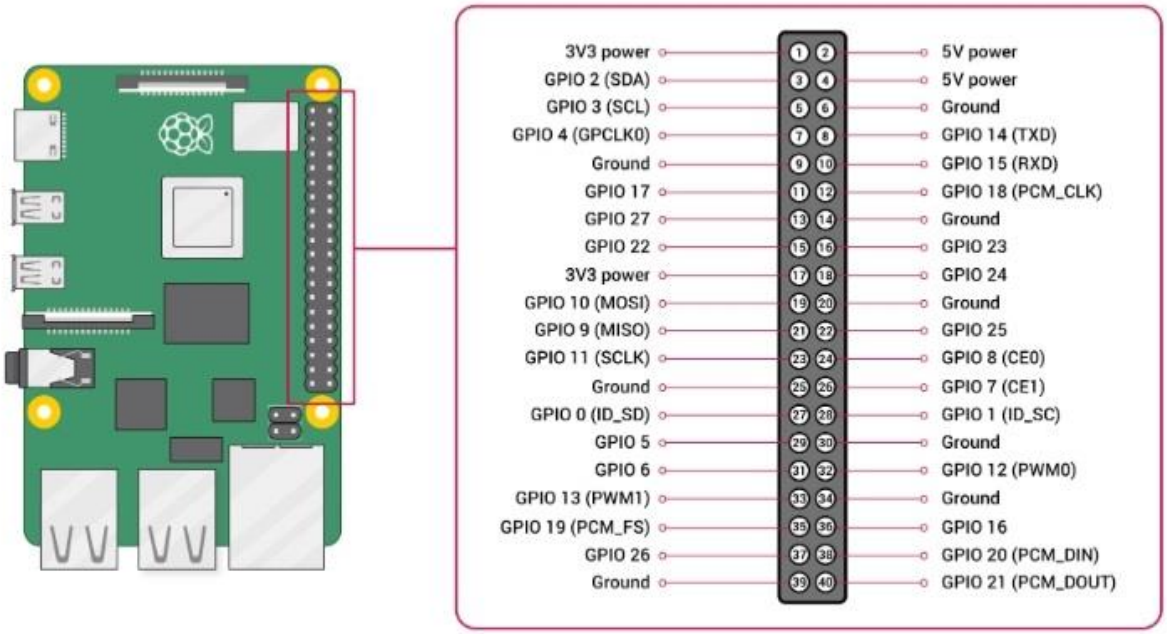
bot = telepot.Bot('Bot Token')

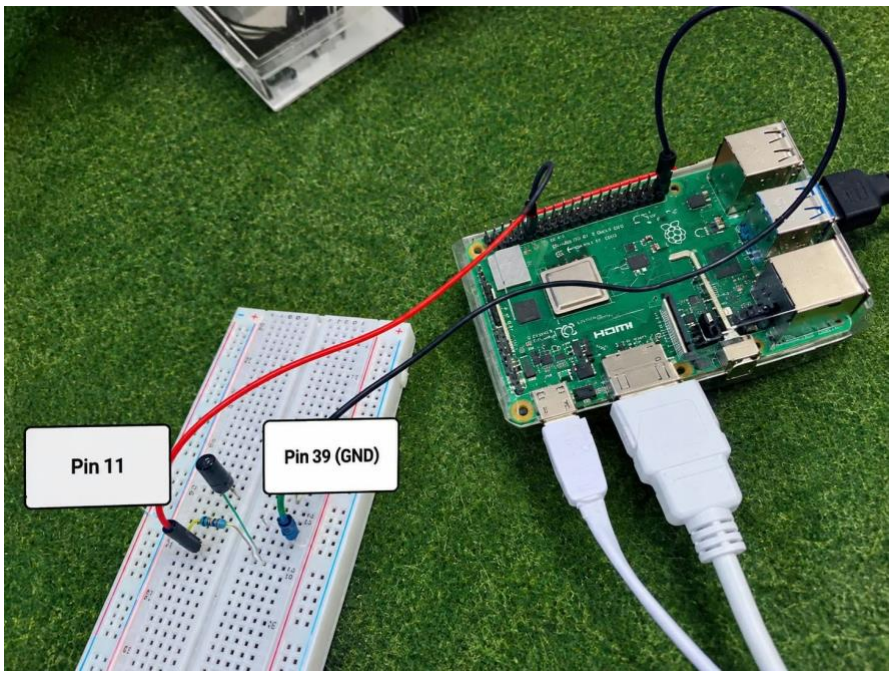
bot.getMe()

If it prints your bot details means everything is correct.

If not then token is wrong.ii. Connection of LED to Raspi.:

- Connect LEDs on breadboard
- Connect resistor to one end of led (to Positive end or anode).
- Connect other end of resistor to GPIO pin on raspberry Pi.
- Connect other end of LED to ground (negative end or cathode).





7. Implementation with Python Code

Question 1: Write a program to control single LED

Code:

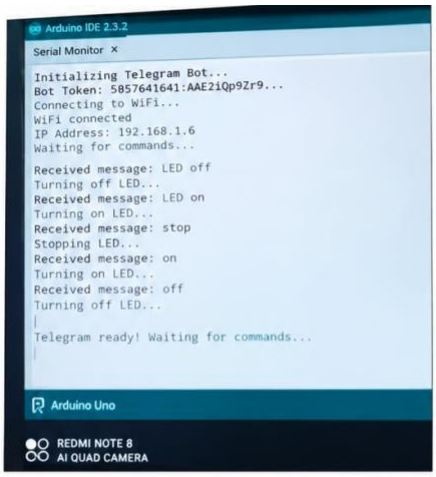
```
import sys
import time
import random
import datetime
import telepot
import RPi.GPIO as GPIO
GPIO.setmode(GPIO.BOARD)
GPIO.setup(11, GPIO.OUT)
def handle(msg):
chat_id = msg['chat'] ['id']
command = msg['text']
try:
print 'Got command:', command
if command == 'on':
bot.sendMessage(chat_id, "LED on")
GPIO.output(11,GPIO.HIGH)
elif command =='off':
bot.sendMessage(chat_id, "LED off")
GPIO.output(11,GPIO.LOW)
elif command == 'stop':
exit()
bot = telepot.Bot('Bot Token')
bot.message_loop(handle)
print 'I am listening...'
```

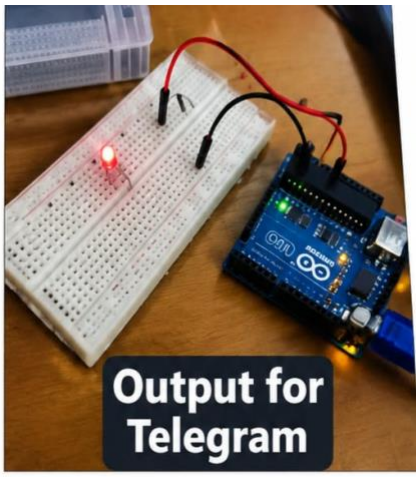
```
while 1:
        time.sleep(10)
except TelegramError:
print ‘’
```

Note: do not forget to replace 'Bot Token' with your token received from telegram

8. Results:

a) For Single LED







PRACTICAL -3

1.Aim:

Interface TM1637 with Raspberry Pi and Display current time.

2.Prior Concepts:

i) Concept of Python Programming

ii) Concept of 7 Segment Displays.

3.New Concepts:

i) Hardware connection of 7 seg with Raspi.

Display current time on 7 segments.

4. Hardware Requirement and PIN Connection.:

i) TM1637 4-digit seven segment Display board.

ii) Jumper wires

Pin Connection

TM1637 PIN	Function	RPi Pin	Function
1	GND(Ground)	6	Ground
2	VCC(+5V Power)	2	5V
3	DIN(Data In)	38	GPIO 20
4	CLK(Clock)	40	GPIO 21

5.Procedure:

Install Libraries of TM1637 as tm1637.py which is driver library. Download and save it in same folder as the main program code.

Write the below code:

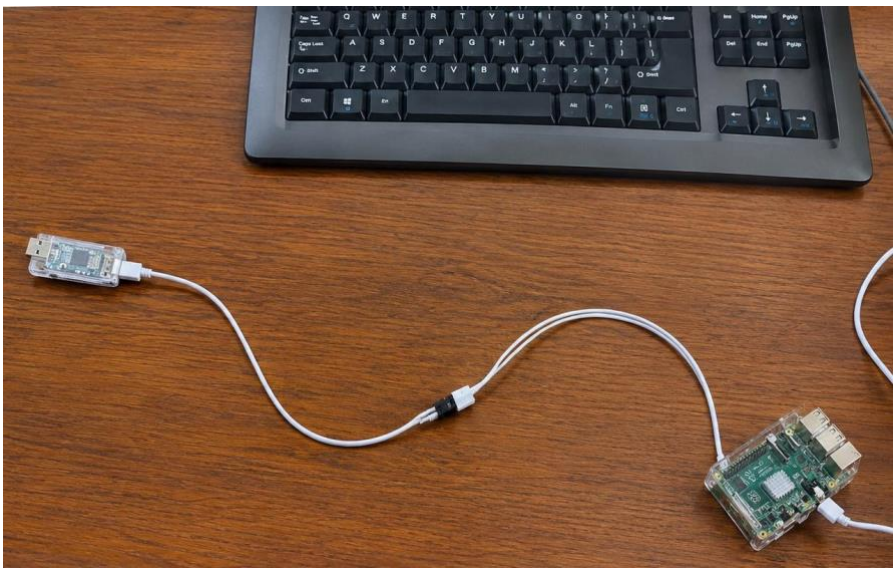
```
#!/usr/bin/python

import time
import datetime
import tm1637 as obj

Display = obj. TM1637(CLK=21, DIO=20, brightness=5.0)
Display.Clear()

while(True):
        now = datetime.datetime.now()
        hour = now.hour
        minute = now.minute
        second = now.second
        Display.Clear()
        val = [(int(hour /10)), (hour % 10), (int(minute / 10)), (minute % 10) ]
Display.Show(val)
        Display.ShowDoublepoint((second % 2))
        time.sleep(0.25)
```

6. Results:



PRACTICAL-4

1.Aim:

Interface GPS module with Raspberry Pi

2.Prior Concepts

i)Concept of Python Programming

ii) Working of GPS Module Sensor

3.New Concepts

i) Hardware connection of GPS Module with Raspi.

4.Hardware Requirement and PIN Connection

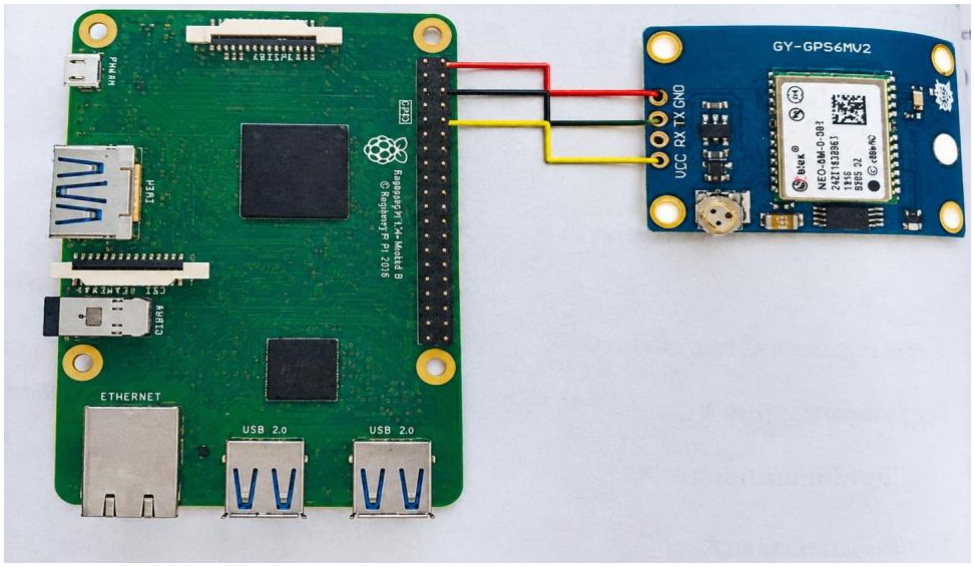
Raspberry Pi 4

ii) GPS Module (e.g., NEO-6M GPS Module)

iii) Jumper Wires

Pin Connection

GPS Module PIN	RPi	Function
GND(Ground)	6	GND(Ground)
VCC(+5V Power)	2	5V
Tx	10	GPIO 15(RXD)
Rx	8	GPIO 14(TXD)



5. Procedure

i) Enable UART on Raspberry Pi by writing below code in command terminal

sudo raspi-config

ii)Navigate:

Interface Options

→ Serial Port

→ Login shell over serial? No

→ Serial port hardware enabled? Yes

iii) Reboot Raspi by writing below code in command terminal

sudo reboot

iv) Verify Serial port by writing below code in command terminal

Is /dev/serial*

Expected Output:

/dev/serial0

/dev/serial1

v) Install GPS Packages, on Raspberry Pi by writing below code in command terminal

sudo apt update

sudo apt install gpsd gpsd-clients python3-gps -y

vi) Stop GPS daemon, on Raspberry Pi by writing below code in command terminal sudo systemctl stop gpsd.socket

sudo systemctl disable gpsd.socket

vii)Start GPS daemon manually, on Raspberry Pi by writing below code in command terminal.

sudo gpsd /dev/serial0-F/var/run/gpsd.sock

viii) Test GPS Data, on Raspberry Pi by writing below code in command terminal

cgps-s

OR

gpsmon

Note: You should see NMEA sentences and GPS coordinates once the module acquires satellites.

Example:

Latitude: 19.0760

Longitude: 72.8777

Altitude: 14.2 m

Time: 12:45:30 UTC

ix) Write the below command to read data from GPS.

pip3 install pyserial pynmea2

Write the below code:

```
import serial
import pynmea2

port = "/dev/serial0"
ser = serial.Serial(port, baudrate=9600, timeout=1)

while True:
    data = ser.readline().decode('ascii', errors='replace')

if data.startswith('$GPGGA'):
    msg = pynmea2.parse(data)
    print("Latitude:", msg.latitude)
```

```
print("Longitude:", msg.longitude)
 print("Altitude :", msg.altitude)
 print("--------------------------")
```

6)Results:

PRACTICAL-5

1. Aim

Interface ADS 1115 with Raspberry Pi to act as oscilloscope.

2. Prior Concepts

i) Concept of Python Programming

ii) Use of Gas Sensor

ii) Concept of Oscilloscope

3. New Concepts

i) Hardware interfacing of Gas Sensor with ADS1115 ADC and Raspi.

ii) Generating waveform of Gas Sensor values.

4. Hardware Requirement and PIN Connection

i) Raspberry Pi 4

ii)ADS1115 ADC

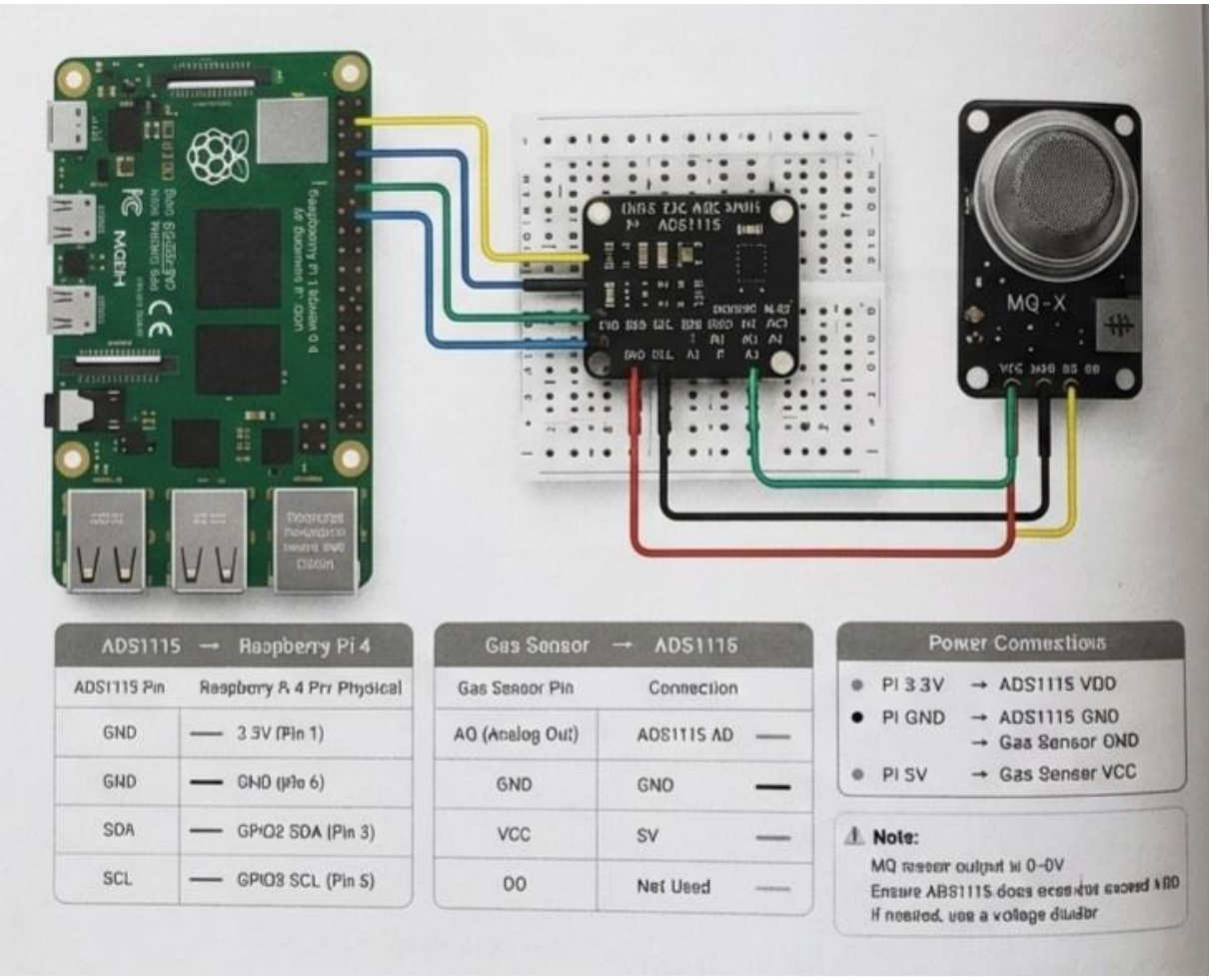
iii) MQ-2 Gas Sensor

iv)Jumper wires

Pin Connection

ADS1115	RPi Pin	Function
VDD(5V)	2	5V
GND	6	GND(GROUND)
SDA	3	GPIO 2(SDA1)
SCL	5	GPIO 3(SCL1)

MQ2 Gas Sensor	ADS1115	RPi Pin	Function
VCC	-	4	5V
GND	-	39	GND(GROUND)
A0	A0	-	-



5. Procedure

Step 1: Enable Raspberry Pi I2C interface

To enable the I2C, from the terminal, run;

sudo raspi-config

When the configuration panels open, select interface options, select I2C and click enable.

Step 2: Update the Raspberry pi

To do this, run below two commands:

sudo apt update

Step 3: Install the Adafruit ADS1115 library for ADC and Matplotlib for plotting of graph.

pip3 install adafruit-circuitpython-ads1x15 matplotlib --break-system-packages

pip3 uninstall -y adafruit-platformdetect adafruit-blinka

sudo reboot

pip3 install --no-cache-dir --upgrade adafruit-platformdetect adafruit-blinka

sudo reboot

Note:

Once all the above configuration command run and connection done then below command should be run.

sudo apt-get install -y i2c-tools python3

Ismod | grep i2c_

i2cdetect -y 1

Write the below code:

```
import time
import board
import busio
import matplotlib.pyplot as plt
from collections import deque
import adafruit_ads1x15.ads1115 as ADS
from adafruit_ads1x15.analog_in import AnalogIn
i2c = busio.I2C(board.SCL, board.SDA)
ads = ADS.ADS1115(i2c)
ads.gain = 1 # ±4.096
chan = AnalogIn(ads, ADS. PO)
x_vals = deque(maxlen = 50)
Y_vals = deque(maxlen = 50)
```

```
pit.ion()
fig, ax = plt.subplots()
line, = ax.plot([],[], label="Gas Sensor Values from ADS1115", marker='o') ax.set_ylim(0,2)
ax.set_xlim(0,100)
ax.set_title('MQ2 Gas Sensor Live Plot')
ax.set_xlabel('Sample')
ax.set_ylabel('Voltage(V)')
ax.legend()

counter = 0

print("Reading Gas Sensor values in Volts...!")
try:
        while True:
        v = chan.voltage
        x_vals.append(counter)
        y_vals.append(v)
        line.set_xdata(x_vals)
        line.set_ydata(y_vals)
        ax.set_xlim(counter-100 if counter > 100 else 0, counter + 1)
        fig.canvas.draw()
        fig.canvas.flush_events()
        time.sleep(1)
        counter+=1

except KeyboardInterrupt:
print("Stopped")
```

6. Results:

PRACTICAL-6

1.Aim

Interface 16x2 LCD with Raspberry Pi using 12C module to display text.

2. Prior Concepts

i) Concept of Python Programming.

ii) LCD function and use.

3. New Concepts

Hardware connection of 16 X 2 LCD with Raspi using I2C Module.

4. Hardware Requirement and PIN Connection.

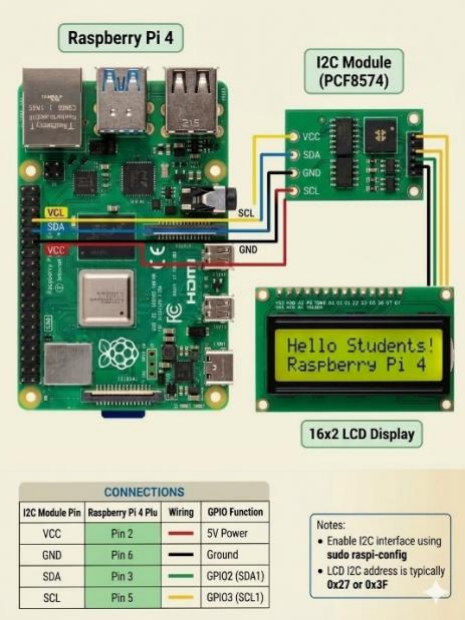
i) Raspberry Pi 4

ii) 12C LCD Module (PCF8574)

iii) Jumper Wires

Pin Connection

LCD I2C Module PIN	RPi Pin	Function
GND(Ground)	6	GND(Ground)
VCC(+5V Power)	2	5V
SDA	3	GPIO 2(SDA1)
SCL	5	GPIO 3(SCL1)



5. Procedure

i) Enable I2C Interface on Raspberry Pi by writing below code in command terminal

sudo raspi-config

ii) Navigate:

Interface Options

→ I2C

→ Enable

iii) Reboot Raspi by writing below code in command terminal

sudo reboot

iv) Install I2C Tools by writing below code in command terminal

sudo apt update

sudo apt install -y i2c-tools python3-smbus

Verify whether LCD is detected

sudo i2cdetect -y 1

Expected Output:

0123456789abcdef

20:-- -- -- -- -- -- 27 -- -- -- -- --

Note: The LCD address is typically Ox27 or Ox3F,

v)Install Install LCD Library on Raspberry Pi by writing below code in command terminal.

pip3 install RPLCD

Write the below code:

```
from RPLCD.i2c import CharLCD
from time import sleep

lcd = CharLCD(
        i2c_expander='PCF8574',
        address=0x27,
        port=1,
        cols=16,
        rows=2
)
I
cd.clear()

lcd.write_string("Hello Students!")
lcd.cursor_pos = (1, 0)
lcd.write_string("Raspberry Pi 4")

sleep(10)

lcd.clear()
```

6. Results:

PRACTICAL-7

1. Aim

Interface RFID module with Raspberry Pi to read and write to RFID tags/cards

2. Prior Concepts

i) Concept of Python Programming.

ii) SPI Communication interface.

3. New Concepts

Interfacing RFID with Raspi.

4. Hardware Required

i) RFID RC522 Reader Module

ii)Few RFID Cards or RFID Tags

iii) Jumper Wires

5. Procedure

Pin Connection

RFID Module PIN	RPi Pin	Function
SDA	24	GPIO 8(SPIO CS0)
SCK	23	GPIO 11(SPIO SCLK)
MOSI	19	GPIO 10(SPIO MOSI)
MISO	21	GPIO 9(SPIO MISO)
GND	6	GND(Ground)
RST	22	GPIO 25
3.5v	1	3.3V

i) Install Python3 on Raspi.

Write the below code in command prompt:

sudo apt install idle3

Setting up Raspberry Pi for Serial Communication

sudo raspi-config

In the "Interfacing Options", select the "Serial" option.

Select "Yes" option.

Finish the process and Reboot the Raspberry Pi.

Following are the terminal command that needs to be executed.

sudo apt update

python3 -m pip install spidev

pip install mfrc522

6. Implementation with Python Code

Code:

```
import RPI.GPIO as GPIO
from mfrc522 import SimpleMFRC522
from time import sleep

reader = SimpleMFRC522()
print("Welcome to RFID Practical")
while True:
```

```
print("")
print ("Choose your Option")
print("1. Write your Tag")
print("2. Read your Tag")
print("3. Delete the content")
print("Press any key to Exit")
print("")
a=input("Enter your option:")

if a=="1":
        text=input("New data: ")
        print("Now Place your Tag to write")
        reader.write(text)
        print("Written")
        sleep(1)

elif a=="2":
        print("Now Place your Tag to read")
        id, text=reader.read()
        print(id)
        print(text)
        sleep(1)

elif a=="3":
        print("Now Place your Tag to Reset")
        reader.write(text)
        print("Data Deleted")
        sleep(1)
```

```
else:
        break

GPIO.cleanup()
```

[Note: now place the RFID cards one by one over RFID module to check the output

7. Results:

PRACTICAL -8

1.Aim

Interface Fingerprint Sensor with Raspberry Pi.

2.Prior Concepts

i) Concept of Python Programming.

ii) SPI Communication interface.

3. New Concepts

Hardware connection of fingerprint sensor.

4. Hardware Required

i) USB to TTL converter.

ii)Fingerprint Module

iii) Jumper Wire

5.Procedure

Connect the finger print module to any one of the USB port of Raspi.

i) Install Python3 on Raspi.

Write the below code in command prompt:

sudo apt install idle3

Setting up Raspberry Pi for Serial Communication

sudo raspi-config

In the "Interfacing Options", select the "UART" option.

Select "Yes" option.

Finish the process and Reboot the Raspberry Pi.

Following are the terminal command that needs to be executed.

sudo apt update

pip install pyfingerprint

6. Implementation with Python Code

Code:

```
#l/usr/bin/env python_
#-- coding: utf-8--

import time
import RPi.GPIO as GPIO
import sys
from threading import Thread
from pyfingerprint.pyfingerprint import PyFingerprint

##############GP|O configuration#######
GPIO.setwarnings(False)
GPIO.cleanup()
GPIO.setmode (GPIO.BCM)
########################################

################### Global Variables ##############
#Initialize Fingerprint Module object
try:
        fingerprint_module = PyFingerprint('/dev/ttyUSBO', 57600,
OxFFFFFFFF,0x00000000)
        if (fingerprint_module.verifyPassword() == False ):
```

```
            raise ValueError('The given fingerprint sensor password is wrong!')
    except Exception as e:
            print('Exception message: '+ str(e))
            exit(1)

######################User function##############################
def delayms(time_in_msec):
    time.sleep( (time_in_msec/1000))
    pass

def delaysec(time_in_sec):
    time.sleep(time_in_sec)

def enrollFingerInDB():
    print ("Enroll your Finger into Fingerprint Database")

    ==========Start===========
try:
    print('Keep your finger...')

    # wait till the person keep finger on module
    while (fingerprint_module.readImage() == False ):
            pass
    # Transform read fingeprint image to characteristics and stores
    charbuffer 1
    fingerprint_module.convertlmage(0x01)

    # Checks for duplicate template in database
    result = fingerprint_module.searchTemplate()
    positionNumber = result[0]
```

```
        if (positionNumber >= 0):
                print('This fingerprint template already exists at position number = '
                +str(positionNumber))


        print('Remove your finger...')
        delaysec(2)


        print('Keep your finger again...')


        ## Wait that finger is read again
        while (fingerprint_module.readImage() == False ):
                pass


        ## Converts read image to characteristics and stores it in charbuffer 2
        fingerprint_module.convertImage(Ox02)


        ## Compares the charbuffers
        if (fingerprint_module.compareCharacteristics() == 0):
                raise Exception('Fingers do not match')


        ## Creates a template
        fingerprint_module.create Template()


        ## Saves template at new position number
        positionNumber = fingerprint_module.storeTemplate()
        print('Finger enrolled successfully!')
        print('New template position #'+ str(positionNumber))
except Exception as e:
        print('Enrollment fail.')
        print('Error:+ str(e))
         ==========End ==========
```

```
        pass


def searchFingerlnDB():
        print ("Search your Finger into Fingerprint Database")
#================Start==================
        try:
                print('Keep your finger...')
                ##Wait that finger is read
                while (fingerprint_module.readImage() == False ):
                        pass
                ##Converts read image to characteristics and stores it in charbuffer 1
                fingerprint_module.convertlmage(Ox01)


                ## Searchs template


                result = fingerprint_module.searchTemplate()


                positionNumber = result[0]
                 accuracyScore = result[1]


                if (positionNumber == -1):
                        print('No fingerprint template match found.')
                else:
                        print('Fingerprint found at position number ='+ str(positionNumbe


        except Exception as e:
                print('Searching fail.')
                print('Error:'+ str(e))
```

```
#==================End========
        pass

def deleteFingerinDB():
        print ("Delete your Finger data from Fingerprint Database")

#==============Start===========

try:
        positionNumber = input('Please enter the template position you want to delete (in range 0 to 1000): ')
        positionNumber = int(positionNumber)
        if (fingerprint_module.deleteTemplate(positionNumber) == True ):
                print('Template deleted successfully.')

except Exception as e:
        print('Deletion fail.')
        print('Error: '+ str(e))

#===========End===========
pass
def clearscreen():
        for i in range(5):
                print ("\n")
                pass
                pass

def clearscreen(no_of_line):
        for i in range(no_of_line):
                print ("\n")
                pass
```

```
pass


def getFingerprintTemplateCount():
        clearscreen(5)
        print('Fingerprint        template , in database = ‘
str(fingerprint_module.getTemplateCount())
str(fingerprint_module.getStorageCapacity()))       +’/
pass


## Tries to initialize the sensor


################## End of User function ##################
def main():
        clearscreen(5)
        choice = 0                  #for choosing the option
        global fingerprint_module #object for fingerprint module


print ("Initialization done...Starting program")
delaysec(1)


while True:                 #1st loop
        clearscreen(25)
        print( "\n\n Choose option in Menu below.\n 1. Enroll Fingerprint.\n Search
Fingerprint.\n 3. Delete Fingerprint.\n")
print (" 4. Get fingerprint record count in Database.\n 5. Exit Program.\n\n")
print (" Enter choice: ")
choice = int(input())


#Enroll finger into Database
if choice == 1:
        enrollFingerInDB()
```

```
        pass


#Search finger into Database.
        if choice == 2:
                searchFingerlnDB()
                pass

#Delete finger into Database.
        if choice == 3:
                deleteFingerlnDB()
                pass

#Get fingerprint templates database information
        if choice == 4:
                getFingerprintTemplateCount()
                pass

        if choice == 5:
                exit(0)
                pass
        delaysec(0.5)
        pass                            #end of 1st loop
pass
if name =='main ':
        try:
                main()
        except KeyboardInterrupt:
                pass
        finally:
                GPIO.cleanup()
```

```
print ("\n\nProgram terminated...")
```

7. Results

PRACTICAL-9

1.Aim

Interface Pi Camera with Raspberry Pi to capture image and video.

2. Prior Concepts

i) Basics of Raspberry Pi and Linux OS.

ii) Python programming fundamentals.

iii) GPIO and peripheral interfacing concepts.

3. New Concepts

i) Working of the Pi Camera Module.

ii) CSI (Camera Serial Interface) communication.

iii) Capturing images using Python.

iv) Recording videos using Python.

v) Using the picamera2 library.

vi) Camera preview and media storage handling.

4. Hardware Required

i) Raspberry Pi Camera Module

ii) Few RFID Cards or RFID Tags

iii)Jumper Wires

5. Procedure

Connect the finger print module to any one of the USB port of Raspi

i)Install Python3 on Raspi.

Write the below code in command prompt:

```
sudo apt install idle3
sudo apt install python3-picamera2 -y
sudo apt install libcamera-apps -y
pip install opencv-python
pip install numpy
```

6. Implementation with Python Code

Code:

```
from picamera2 import Picamera2
from time import sleep

#Initialize camera
pícam2 = Picamera2()

#Start camera
picam2.start()
```

```
# Allow camera to warm up
sleep(2)
#-----------------------------
#Capture Image
#-----------------------------
picam2.capture_file("captured_image.jpg")
print("Image Captured Successfully")

#---------------------------
# Record Video
#---------------------------
print("Video Recording Started")

#Start video recording
picam2.start_recording("captured_video.h264")

# Record for 10 seconds
sleep(10)

#Stop recording
picam2.stop_recording()

print("Video Recorded Successfully")

#Stop camera
picam2.stop()

print("Program Completed")
```

7. Results:

PRACTICAL-10

1.Aim

Design loT based home automation system using Raspberry Pi.

2. Prior Concepts

i) Concept of Python Programming

ii) Working of relay modules

iii) Basic networking concepts

3. New Concepts

i) loT-based remote appliance control

ii)Interfacing relay modules with Raspberry Pi

iii) Web-based appliance automation

iv) Flask web server programming

v)GPIO control using Python

vi) Smart home automation concepts

4. Hardware Required

i) Relay Module (2-channel)

ii)Blub

iii)DC Fan

iv) Jumper Wires

5.Procedure

Pin Connection

i) Install Python3 on Raspi.

Write the below code in command prompt:

sudo apt install idle3

pip install flask

pip install RPi.GPIO

6. Implementation with Python Code

Code:

```
from flask import Flask
import RPi.GPIO as GPIO
```

```
app = Flask(name)

#GPIO Setup
GPIO.setmode(GPIO.BCM)

LIGHT = 17
FAN = 27
GPIO.setup(LIGHT, GPIO.OUT)
GPIO.setup (FAN, GPIO.OUT)

GPIO.output(LIGHT, GPIO.LOW)
GPIO.output(FAN, GPIO.LOW)

@app.route('/')
def home():
        return "'
        <h1>loT Home Automation</h1>

        <h3>Light </h3>
        <a href="/light/on">ON</a><br><br>
        <a href="/light/off">OFF</a><br><br>
        <h3>Fan</h3>
        <a href="/fan/on">ON</a><br><br>
        <a href="/fan/off">OFF</a>
        ‘‘
@app.route("/light/on')
def light_on():
GPIO.output(LIGHT, GPIO.HIGH)
return "Light ON"
```

```
@app.route('/light/off')
def light_off():
        GPIO.output(LIGHT, GPIO.LOW)
        return "Light OFF"

@app.route('/fan/on')
def fan_on():
        GPIO.output(FAN, GPIO.HIGH)
        return "Fan ON"

@app.route('/fan/off')
def fan_off():
        GPIO.output (FAN, GPIO.LOW)
        return "Fan OFF"

if name ==' _main':
app.run(host='0.0.0.0', port=5000)
```

7. Execute the above Program:

i)Run the above Program

ii) Find Raspi IP Address, by executing below command on command prompt:

hostname -I

Eg: 192.168.1.10

iii) Open Web Interface on mobile or laptop and enter the IP address in browser

Eg: http://192.168.1.10:5000

8. Result:

PRACTICAL – 11

1. Aim

Interface Fingerprint Sensor and 16X2 LCD with Raspberry and display message Fingerprint matched or unmatched.

2. Prior Concepts

i) Concept of Python Programming.

ii) Knowledge of UART serial communication.

iii) Basics of I2C communication.

iv) Basic LCD display programming.

3. New Concepts

i) Interfacing a biometric fingerprint sensor with Raspberry Pi.

ii) Using UART communication for fingerprint sensor data exchange.

iii) Interfacing a 16×2 LCD through the I2C protocol.

iv) Developing a complete fingerprint-based access verification system.

4. Hardware Required

i) Fingerprint Sensor (R307, AS608, etc.)

ii) 16×2 LCD with I2C module

iii) Jumper Wires

5. Procedure

Pin Connection

i) Fingerprint Sensor → Raspberry Pi

Fingerprint Sensor	Raspberry Pi
VCC	5V
GND	GND
TX	GPIO15 (RXD)
RX	GPIO14 (TXD)

ii) I2C LCD → Raspberry Pi

LCD I2C	Raspberry Pi
VCC	5V
GND	GND
SDA	GPIO2 (SDA)
SCL	GPIO3 (SCL)

Input image

i) Install Python3 on Raspi.

Write the below code in command prompt:

sudo apt install idle3

Enable UART on Raspberry Pi:

sudo raspi-config

Navigate to:

Interface Options → Serial Port

Login Shell? → No

Serial Hardware? → Yes

Reboot:

sudo reboot

Enable I2C on Raspberry Pi :

sudo raspi-config

Navigate to:

Interface Options → I2C → Enable

Install Libraries

sudo apt update
sudo apt install python3-pip -y
pip3 install pyfingerprint
pip3 install RPLCD
pip3 install RPLCD-i2c

6. Implementation with Python Code

Code :

```
from pyfingerprint.pyfingerprint import PyFingerprint
from RPLCD.i2c import CharLCD
from time import sleep
```

```
# LCD configuration
lcd = CharLCD('PCF8574', 0x27)
lcd.clear()
lcd.write_string('Place Finger')

try:

    # Fingerprint sensor
    f = PyFingerprint('/dev/serial0', 57600, 0xFFFFFFFF,
                      0x00000000)

    if not f.verifyPassword():
        raise ValueError ('Fingerprint sensor password incorrect')

except Exception as e:

    print ('Sensor initialization failed!')

    print (e)

    exit (1)

while True:

    lcd.clear ()

    lcd.write_string ('Place Finger')

    # Wait for finger
    while not f.readImage():

        pass

    # Convert image
```

```
f.convertImage(0x01)

# Search template

result = f.searchTemplate()

positionNumber = result[0]

lcd.clear ()

if positionNumber >= 0:

    lcd.write_string('Fingerprint')

    lcd.crlf ()

    lcd.write_string('Matched')

    print ("Fingerprint Matched")

else:

    lcd.write_string('Fingerprint')

    lcd.crlf()

    lcd.write_string('Unmatched')

    print("Fingerprint Unmatched")

sleep(3)
```

7. Result

PRACTICAL – 12

1. Aim

Simulate door lock by interfacing fingerprint sensor and relay module with raspberry Pi. (This practical instead of door, bulb is used to show the output).

2. Prior Concepts

i) Basic Raspberry Pi architecture and GPIO programming.

ii) Knowledge of UART serial communication.

iii) Basics of biometric authentication.

iv) Relay module operation and switching principles.

3. New Concepts

i) Interfacing a biometric fingerprint sensor with Raspberry Pi.

ii) Controlling a relay based on authentication results.

iii) Integration of sensing, processing, and actuation in an embedded system.

4. Hardware Required

i) Fingerprint Sensor (R307, AS608, etc.)

ii) Relay Module (5V)

iii) Bulb

iv) Jumper Wires

5. Procedure

Pin Connection

i) Fingerprint Sensor → Raspberry Pi

Fingerprint Sensor	Raspberry Pi
VCC	5V
GND	GND
TX	GPIO15(RXD)
RX	GPIO14(TXD)

ii) Relay Module to Raspberry Pi

Relay Module	Raspberry Pi
VCC	5V
GND	GND
IN	GPIO18

iii) Bulb Connection

Bulb	Relay Module
Positive Terminal	No (Normally Open)
Negative Terminal	GND/Power Return

Input IMAGE

i) Install Python3 on Raspi.

Write the below code in command prompt:

sudo apt install idle3

Enable UART on Raspberry Pi:

sudo raspi-config

Navigate to:

Interface Options → Serial Port

Login Shell? → No

Serial Hardware? → Yes

Reboot:

sudo reboot

Install Libraries

sudo apt update

sudo apt install python3-pip -y

pip3 install pyfingerprint

6. Implementation with Python Code

Code:

```
from pyfingerprint.pyfingerprint import PyFingerprint

import RPi.GPIO as GPIO

from time import sleep

# Relay connected to GPIO18
RELAY_PIN = 18

GPIO.setwarnings(False)
GPIO.setmode(GPIO.BCM)

GPIO.setup(RELAY_PIN, GPIO.OUT)
```

```
# Initially bulb OFF
GPIO.output(RELAY_PIN, GPIO.LOW)

# Initialize Fingerprint Sensor
try:

    fp = PyFingerprint(
        '/dev/serial0',
        57600,
        0xFFFFFFFF,
        0x00000000
    )

    if not fp.verifyPassword():
        raise ValueError('Fingerprint sensor password incorrect')

except Exception as e:

    print("Fingerprint Sensor Initialization Failed")

    print(e)

    exit(1)

print("= = = =
```

```
    print("\nPlace Finger on Sensor...")

    # Wait until finger is placed
    while not fp.readImage():
        pass

    # Convert image to characteristics
    fp.convertImage(0x01)

    # Search fingerprint template
    result = fp.searchTemplate()

positionNumber = result[0]

if positionNumber >= 0:

    print("Fingerprint Matched")
    print("Bulb ON")

    # Turn ON Bulb through Relay
    GPIO.output(RELAY_PIN, GPIO.HIGH)

    # Keep bulb ON for 5 seconds
    sleep(5)

    # Turn OFF Bulb
    GPIO.output(RELAY_PIN, GPIO.LOW)

    print("Bulb OFF")

else:

    print("Unauthorized User")
    print("Access Denied")
```

```
    print("Bulb Remains OFF")

    GPIO.output(RELAY_PIN, GPIO.LOW)

    sleep(2)
```

7. Result:

PRACTICAL – 13

1. Aim

Implement basic Publisher – Subscriber using MQTT. Publish messages using mqtt. Subscribe and control LED.

2. Prior Concepts

i) Basic Raspberry Pi architecture and GPIO programming.

ii) Client–Server communication.

iii) Command-line operations in Linux.

3. New Concepts

i) MQTT publish-subscribe communication model.

ii) Working of MQTT broker, publisher and subscriber.

iii) Using Paho MQTT library in Python.

iv) Event-driven programming using MQTT callbacks.

v) Building cloud-ready IoT applications.

4. Hardware Required

i) LED

ii) 220 Ω Resistor

iii) Jumper Wires

5. Procedure

Pin Connection

LED Connection

LED Pin	**Raspberry Pi 4**
Anode (+)	GPIO18 (Pin 12) through 220Ω resistor
Cathode (-)	GND (Pin 6)

Input IMAGE

i) Install Python3 on Raspi.

Write the below code in command prompt:

sudo apt install idle3

Install MQTT Broker:

sudo apt update
sudo apt install mosquitto mosquitto-clients -y

Enable broker:

sudo systemctl enable mosquitto
sudo systemctl start mosquitto

Install Paho MQTT:

pip3 install paho-mqtt

6. Implementation with Python Code

Subscriber Code (LED Control):

```
import paho.mqtt.client as mqtt
import RPi.GPIO as GPIO

LED = 18

GPIO.setmode(GPIO.BCM)
GPIO.setup(LED, GPIO.OUT)

BROKER = "localhost"
TOPIC = "iot/led"

def on_connect(client, userdata, flags, rc):
    print("Connected")
    client.subscribe(TOPIC)

def on_message(client, userdata, msg):

    message = msg.payload.decode()

    print("Received:", message)

    if message.upper() == "ON":
        GPIO.output(LED, GPIO.HIGH)
        print("LED ON")

    elif message.upper() == "OFF":
        GPIO.output(LED, GPIO.LOW)
        print("LED OFF")
```

```
client = mqtt.Client()

client.on_connect = on_connect
client.on_message = on_message

client.connect(BROKER, 1883, 60)

client.loop_forever()
```

Publisher Code:

```
import paho.mqtt.client as mqtt

broker = "localhost"
topic = "iot/led"

client = mqtt.Client()

client.connect(broker, 1883, 60)

while True:

    msg = input("Enter ON/OFF : ")

    client.publish(topic, msg)

    print("Published :", msg)
```

7. Execution Step

Terminal 1: Run Subscriber

python3 subscriber.py

Terminal 2: Run Publisher

python3 publisher.py

8. Result

Subscriber Terminal:

Connected
Received: ON
LED ON
Received: OFF
LED OFF

Publisher Terminal:

text
Enter ON/OFF : ON
Published : ON
Enter ON/OFF : OFF
Published : OFF

PRACTICAL – 14

1. Aim

Design a web dashboard using Flask. Control relay module via web.

2. Prior Concepts

i) Concept of Python Programming.

ii) Working of relay modules.

iii) Basic networking concepts.

3. New Concepts

i) IoT-based remote appliance control.

ii) Interfacing relay modules with Raspberry Pi.

iii) Web-based appliance automation.

iv) Flask web server programming.

v) GPIO control using Python.

vi) Smart home automation concepts.

4. Hardware Required

i) Relay Module (2-channel)

Input IMAGE

ii) Bulb

iii) DC Fan

iv) Jumper Wires

5. Procedure

Pin Connection

Input IMAGE

i) Install Python3 on Raspi.

Write the below code in command prompt:

sudo apt install idle3

pip install flask

pip install RPi.GPIO

6. Implementation with Python Code

Code:

```
from flask import Flask

import RPi.GPIO as GPIO

app = Flask(__name__)

# GPIO Setup
GPIO.setmode(GPIO.BCM)

LIGHT = 17

FAN = 27

GPIO.setup(LIGHT, GPIO.OUT)

GPIO.setup(FAN, GPIO.OUT)

GPIO.output(LIGHT, GPIO.LOW)

GPIO.output(FAN, GPIO.LOW)



@app.route('/')
def home():
    return """
    <h1>IoT Home Automation</h1>
```

```
    <h3>Light</h3>

    <a href="/light/on">ON</a><br><br>

    <a href="/light/off">OFF</a><br><br>

    <h3>Fan</h3>

    <a href="/fan/on">ON</a><br><br>

    <a href="/fan/off">OFF</a>
    """


@app.route('/light/on')
def light_on():
    GPIO.output(LIGHT, GPIO.HIGH)
    return "Light ON"


@app.route('/light/off')
def light_off():
    GPIO.output(LIGHT, GPIO.LOW)
    return "Light OFF"


@app.route('/fan/on')
def fan_on():
    GPIO.output(FAN, GPIO.HIGH)
    return "Fan ON"


@app.route('/fan/off')
def fan_off():
```

```
    GPIO.output(FAN, GPIO.LOW)
    return "Fan OFF"


if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

7. Execute the Above Program

i) Run the above Program.

ii) Find Raspi IP Address, by executing below command on command prompt:

hostname -I

Eg: 192.168.1.10

iii) Open Web Interface on mobile or laptop and enter the IP address in browser.

Eg: http://192.168.1.10:5000

8. Result

The web dashboard using Flask is designed successfully to control the relay module via the web.

PRACTICAL – 15

1. Aim

Create two factor authentication system by interfacing fingerprint sensor, RFID reader with Raspberry Pi.

2. Prior Concepts

i) Concept of Python Programming.

ii) Basics of serial (UART) communication.

iii) Basics of SPI communication.

iv) Basics of I2C communication.

v) Understanding of biometric systems.

3. New Concepts

i) Designing a Two-Factor Authentication (2FA) system.

ii) Integrating multiple authentication devices in a single application.

iii) Displaying authentication status on I2C LCD.

4. Hardware Required

i) MFRC522 RFID Reader

ii) R307/AS608 Fingerprint Sensor

iii) 16×2 LCD with I2C (PCF8574)

iv) Jumper Wires

5. Procedure

Pin Connection

RFID Pin	**Raspberry Pi Pin**
SDA (SS)	GPIO8 (CE0)
SCK	GPIO11
MOSI	GPIO10
MISO	GPIO9
RST	GPIO25

GND	GND
3.3V	3.3V

Fingerprint	**Raspberry Pi**
VCC	5V
GND	GND
TX	GPIO15 (RXD)
RX	GPIO14 (TXD)

LCD	**Raspberry Pi**
VCC	5V
GND	GND
SDA	GPIO2
SCL	GPIO3

Input IMAGE

i) Install Python3 on Raspi.

Write the below code in command prompt:

sudo apt install idle3

Enable following Interfaces:

sudo raspi-config

SPI -> Enable

I2C -> Enable

UART -> Enable

sudo reboot

Install Libraries

sudo apt update
pip3 install pyfingerprint
pip3 install mfrc522
pip3 install RPLCD
pip3 install RPLCD-i2c
pip3 install spidev
pip3 install RPi.GPIO

6. Implementation with Python Code

Code:

```
from pyfingerprint.pyfingerprint import PyFingerprint
from mfrc522 import SimpleMFRC522
from RPLCD.i2c import CharLCD
import RPi.GPIO as GPIO
from time import sleep

# ------------- LCD -------------
lcd = CharLCD('PCF8574', 0x27)
```

```
# ---------- RFID ----------
reader = SimpleMFRC522()

AUTHORIZED_UID = 123456789012

# ---------- Buzzer ----------

BUZZER = 18

GPIO.setwarnings(False)
GPIO.setmode(GPIO.BCM)
GPIO.setup(BUZZER, GPIO.OUT)

# ---------- Fingerprint ----------

try:
    finger = PyFingerprint(
        '/dev/serial0',
        57600,
        0xFFFFFFFF,
        0x00000000
    )

    if not finger.verifyPassword():
        raise ValueError()

except Exception:
    print("Fingerprint Sensor Error")
    exit()

# ---------- Functions ----------

def buzzer_beep(times):
    for i in range(times):
```

```
GPIO.output(BUZZER, 1)

sleep(0.2)

GPIO.output(BUZZER, O)

sleep(0.2)

def Icd_msg(line1, line2=""):

Icd.clear()

Icd.write_string(line1)

if line2:

Icd.crif()

Icd.write_string(line2)

#--------------------Main Program-------------

while True:

Icd_msg("Scan RFID Card")

try:

uid, text = reader.read()

if uid == AUTHORIZED_UID:

Icd_msg("RFID Matched")

buzzer_beep(1)

sleep(2)

Icd_msg("Place Finger")
```

```
while not finger.readImage():
finger.convertImage(0x01)
result = finger.searchTemplate()
position Number = result[0]
if position Number >= 0:
lcd_msg("Fingerprint", "Matched")
buzzer_beep(2)
sleep(2)
lcd_msg("ACCESS", "GRANTED")
print("ACCESS GRANTED")
else:
lcd_msg("Fingerprint", "Unmatched")
buzzer_beep(4)
sleep(2)
lcd_msg("ACCESS", "DENIED")
print("Fingerprint Failed")
else:
lcd_msg("RFID", "Unmatched")
buzzer_beep(4)
sleep(2)
```

```
lcd_msg("ACCESS", "DENIED")

print("Unauthorized RFID")

except KeyboardInterrupt:

GPIO.cleanup()

Break
```

7. Result:

PRACTICAL – 16

1. Aim

Create a motion detector by interfacing camera and 16X2 display with Raspberry Pi, Print the appropriate message on the display.

2. Prior Concepts

i) Concept of Python Programming.

ii) Basics of camera operation.

iii) Basics of 12C communication.

iv) Interfacing 16x2 LCD with Raspberry Pi.

v) Basic image processing concepts.

3. New Concepts

i) Motion detection using computer vision.

ii) Raspberry Pi Camera interfacing.

iii) Frame differencing technique.

iv) Integration of camera and LCD modules.

4. Hardware Required

i) Raspberry Pi Camera Module V2/V3

ii) 16×2 LCD with 12C (PCF8574)

iii) Jumper Wires

5. Procedure

Pin Connection

Camera Module: Connect the Raspberry Pi Camera Module to the CSI Camera Port of Raspberry Pi 4.

12C LCD Connections to Raspi:

LCD (12C)	Raspberry Pi 4
VCC	5V (Pin 2)
GND	GND (Pin 6)
SDA	GPIO2 SDA (Pin 3)
SCL	GPIO3 SCL (Pin 5)

Input IMAGE

1) Install Python3 on Raspi.

Write the below code in command prompt:

sudo apt install idle3

Enable following Interfaces:

sudo raspi-config

12C -> Enable

Camera -> Enable

sudo reboot

Install Libraries

sudo apt update

sudo apt install python3-opencv -y

pip3 install RPLCD

pip3 install smbus2

pip3 install picamera2

6. Implementation with Python Code

Code:

```
from picamera2 import Picamera?

import cv.2

import numpy as np

import time

from RPLCD.i2c import CharLCD

Icd = CharLCD(

PCF8574',
```

```
0x27,
cols=16,
rows 2
)
pcam2 Picamera2()
config = picam2.create_preview_configuration(
main=["size": (640,480))
pram2.configure(config)
picam2.start()
time.sleep(2)
frame1 = picam2.capture_array()
frame1 = cv2.cvtColor(frame1, cv2.COLOR_BGR2GRAY)
while True:
frame2 = picam2.capture_array()
frame2_gray = cv2.cvtColor(
frame2,
cv2.COLOR_BGR2GRAY
)
diff = cv2.absdiff(
frame1,
```

```
frame2_gray

)

threshold= cv2.threshold(

diff,

25,

255,

cv2.THRESH_BINARY

) [1]

motion_pixels = cv2.countNonZero(

threshold

)

lcd.clear()

if motion_pixels > 5000:

print("Motion Detected")

lcd.write_string(

"Motion Detected"

)

else:

print("No Motion")

lcd.write_string(
```

"No Motion"

)

frame1 = frame2_gray

time.sleep(1)

7. Result:

PRACTICAL-17

1. Aim

push-button and an LED with a web dashboard. The flow must allow users to toggle the LED from the dashboard and via the physical button. A button press should force the LED ON Design and implement a Node-RED flow on Raspberry Pi that integrates a physic for 5 seconds and then automatically restore the LED to the dashboard's last state.

2. Prior Concepts

i) Concept of Python Programming

ii) LED interfacing with Raspberry Pi

iii) Push button switch operation

iv) Web browser operation

v) Event-driven programming basics

3. New Concepts

i) Node-RED visual programming.

ii) Flow-based loT application development.

iii) Web-based device control.

iv) Dashboard and hardware synchronization.

v) Timer and delay node implementation.

4. Hardware Required

i) Push Button Switch

ii) LED

iii) Jumper Wires

5. Procedure

Pin Connection

Camera Module: Connect the Raspberry Pi Camera Module to the CSI Camera Port of Raspberry Pi 4

LED Connections to Raspi:

Component	Raspberry Pi 4
LED Anode (+)	GPIO18 (Pin 12) through 220 resistor
LED Cathode (-)	GND (Pin 6)

Input IMAGE

i) Install Python3 on Raspi.

Write the below code in command prompt:

sudo apt install idle3

Install Node-RED Dashboard:

cd ~/.node-red

npm install node-red-dashboard

Restart Node-RED:

node-red-stop

node-red-start

6. Execution

Now open Node Red in browser:

http://<RaspberryPi-IP>:1880

PRACTICAL-18

1.Aim

Write a program to read the CPU%, Memory% and CPU temperature of Raspberry Pi,

2. Prior Concepts

i) Linux operating system fundamentals

ii) Processor and memory concepts.

3. New Concepts

i) System performance monitoring.

ii) CPU utilization measurement.

iii) Memory utilization analysis.

iv) Using the psutil library.

v) Health monitoring of lot devices.

4. Software Required

i) psutil Library

5. Concept:

CPU Utilization: CPU utilization indicates the percentage of processor resources currently in use.

0% →Idle Processor

100% →Fully Utilized Processor

Vemory Utilization: Memory utilization represents the percentage of RAM currently stupied by running programs and processes.

DU Temperature: CPU temperature helps determine the thermal condition of the processor. Excessive temperature may affect performance and reliability.

Typical Raspberry Pi 4 temperatures:

Temperature	Status
35°C-55°C	Normal
55°C-70°C	Moderate Load
70°C-80°C	High Load
>80°C	Thermal Throttling Possible

6. Procedure

i) Install Python3 on Raspi.

Write the below code in command prompt:

Sudo apt install idle3

Install Libraries

sudo apt update

pip3 install psutil

7. Main Program

```
import psutil
import os
import time
while True:
cpu_percent = psutil.cpu_percent(interval=1)
memory_percent = psutil.virtual_memory().percent
temp = os.popen(
"vcgencmd measure_temp"
).readline()
cpu_temp = temp.replace(
"temp=",
""
).replace(
"C\n",
""
```

```
)
print("\n----------------------")
print(" Raspberry Pi 4 Monitoring")
print("----------------------")
print(
f'CPU Usage: {cpu_percent)%"
)
print(
f'Memory Usage: {memory_percent)%"
)
print(
f'CPU Temperature: {cpu_temp} °C"
)
time.sleep(2)
```

8. Result:

Input IMAGE

```
Raspberry Pi 4 Monitoring
-------------------------
CPU Usage: 18.5%
Memory Usage: 42.3%
```

CPU Temperature: 49.8 °C

Raspberry Pi 4 Monitoring

CPU Usage: 24.1%

Memory Usage: 43.7%

CPU Temperature: 50.1 °C

PRACTICAL-19

1. Aim

Design and implement an embedded access-control prototype that grants entry only when either a whitelisted RFID tag is detected or a legitimate fingerprint is verified, provides real-time visual feedback on an LCD, indicates status via an LED, and pulses a relay active-LOW) to unlock for a fixed interval. The system must log all events (time, method, esult) for auditability.

2. Prior Concepts

i) Concept of Python Programming

ii) Basics of RFID technology

iii) Basics of biometric authentication

vi) SPI communication protocol

v) UART communication protocol

vi) 12C communication protocol

vii) Relay interfacing basics

3. New Concepts

i) Development of a complete embedded access-control system.

ii) Integration of multiple authentication methods (RFID and Fingerprint).

iii) Event-driven access control system design.

4. Hardware Required

i) MFRC522 RFID Reader

ii) R307/AS608 Fingerprint Sensor

iii) 16x2 LCD (I2C)

iv) Relay Module (Active-LOW)

5. Procedure

Pin Connection

RFID (MFRC522) ---- Raspi:

MFRC522	Raspberry Pi
SDA	GPIO8
SCK	GPIO11
MOSI	GPIO10
MISO	GPIO9
RST	GPIO25
3.3V	3.3V
GND	GND

Fingerprint Sensor to Raspi:

Sensor	Raspberry Pi
VCC	5V
GND	GND
TX	GPIO15
RX	GPIO14

LCD 12C to Raspi:

LCD	Raspberry Pi
VCC	5V
GND	GND
SDA	GPIO2
SCL	GPIO3

LED to Raspi :

LED	Raspberry Pi
Anode	GPIO17 (via 2200)
Cathode	GND

Active Low Relay to Raspi:

Relay	Raspberry Pi
IN	GPIO27
VCC	5V
GND	GND

Note: Relay Logic

PD27= LOW -> Door Unlock

GP027 HIGH -> Door Locked

Input IMAGE

i) Install Python3 on Raspi.

Write the below code in command prompt:

sudo apt install idle3

Install Libraries

pip3 install mfrc522

pip3 install pyfingerprint

pip3 install RPLCD

pip3 install RPLCD-12c

pip3 install spidev

6. Implementation with Python Code

Code:

```
import time

import csv

from datetime import datetime

import RPI.GPIO as GPIO

from mfrc522 import SimpleMFRC522
```

```
from pyfingerprint.pyfingerprint import PyFingerprint

from RPLCD.i2c import CharLCD

# ==========================================

#CONFIGURATION

# ==========================================

LED_PIN = 17

RELAY_PIN = 27                    #Active-LOW relay

UNLOCK_TIME = 5                   #seconds

LOG_FILE = "access_log.csv"

# Replace with your RFID UID(s)

AUTHORIZED UIDS = [

123456789012,

987654321098

]

# ==========================================

#GPIO SETUP

# ==========================================

GPIO.setwarnings(False)

GPIO.setmode (GPIO.BCM)

GPIO.setup(LED_PIN, GPIO.OUT)
```

```
GPIO.setup(RELAY_PIN, GPIO.OUT)

GPIO.output(LED_PIN, GPIO.LOW)

#Relay OFF (locked)

GPIO.output(RELAY PIN, GPIO.HIGH)

# = = =
```

```
try:
fp = PyFingerprint(
'/dev/serialo',
57600,
0xFFFFFFFF
0x00000000
)
if not fp.verify Password():
raise ValueError(
'Fingerprint sensor password incorrect'
)
wept Exception as e:
print("Fingerprint initialization failed")
print(e)
Icd.clear()
icd.write_string("FP Init Error")
exit()
# = = =
```

```
def lcd_msg(line1="", line2=""):

lcd.clear()

lcd.write_string(str(line1)[:16])

if line 2:

lcd.crlf()

lcd.write_string(str(line2)[:16])

# = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = =

#LOGGING

# = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = =

def log_event(method, identifier, result):

timestamp = datetime.now().strftime(

"%Y-%m-%d %H:%M:%S"

)

with open(LOG_FILE, "a", newline="") as file:

writer = csv.writer(file)

writer.writerow([

timestamp,

method,

identifier,

result
```

```
])
#==========================================
#ACCESS FUNCTIONS
#==========================================
def unlock_door():
GPIO.output(RELAY_ output (RELAY_PIN, GPIO.LOW)
time.sleep(UNLOCK_TIME)
GPIO.output(RELAY_PIN, GPIO.HIGH)
def access_granted(method, identifier):
print(
f"[GRANTED] [method): (identifier)"
)
lcd_msg(
"ACCESS",
"GRANTED"
)
GPIO.output(
LED_PIN,
GPIO.HIGH
)
```

```
log event(
method,
identifier,
"GRANTED"
)
unlock_door()
GPIO.output
LED_PIN,
GPIO.LOW
)
time.sleep(1)
def access_denied(method, identifier):
print(
f"[DENIED] (method): (identifier)"
)
lcd_msg(
"ACCESS",
"DENIED"
)
log_event(
```

```
method,

identifier,

"DENIED"

)

time.sleep(2)

# ==========================================

# RFID CHECK

# ==========================================

def check_rfid():

try:

uid, text = reader.read_no_block()

If uld is None:

return False

print("RFID UID:", uid)

if uid in AUTHORIZED_UIDS:

Icd_msg(

"RFID VERIFIED",

"WELCOME"

)

access_granted(
```

```
"RFID",
Uid
)
else:
lcd_msg(
"RFID INVALID",
str(uid)
)
access_denied(
"RFID",
uid
)
return True
Except Exception as e:
print("RFID Error:", e)
return False
# ==========================================
#FINGERPRINT CHECK
# ==========================================
def check_fingerprint():
```

```
try:
if not fp.readImage():
return False
fp.convertImage(0x01)
result = fp.searchTemplate()
position = result[0]
if position >= 0:
lcd_msg(
"FINGER MATCH",
f"ID:{position}"
)
access_granted(
"FINGERPRINT",
Position
)
else:
Icd_msg(
"NO MATCH",
"UNKNOWN"
)
```

```
access_denied(

"FINGERPRINT",

"UNKNOWN"

)

return True

except Exception as e:

print("Fingerprint Error:", e)

return False

# ==========================================

#MAIN

# ==========================================

def create_log_file():

try:

with open(

LOG_FILE,

"x",

newline=""

) as file:

writer = csv.writer(file)

writer.writerow([
```

```
"Timestamp",
"Method",
"Identifier",
"Result"
])
except FileExists Error:
pass
def main():
create_log_file()
lcd_msg(
"ACCESS CONTROL",
"Ready"
)
print(
"System Ready..."
)
try:
Icd_msg(
"Scan Card or",
"Place Finger"
```

```
)
rfid_detected = check_rfid()
if rfid_detected:
continue
finger_detected = check_fingerprint()
if finger_detected:
continue
time.sleep(0.2)
except KeyboardInterrupt:
print("Exiting...")
finally:
lcd.clear()
GPIO.output(
RELAY_PIN,
GPIO.HIGH
)
GPIO.output(
LED_PIN,
GPIO.LOW
)
```

```
GPIO.cleanup()

# = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = = =

if_name__ = = "__main__":

main()
```

7. Result :

PRACTICAL-20

1. Aim

install Windows IoT 10 core on Raspberry pi and run the demo program.

2. Prior Concepts

i) Basic computer architecture and operating systems.

ii) Windows operating system basics.

iii) Microprocessor and microcontroller fundamentals.

iv) Basic networking concepts (IP address, Ethernet, Wi-Fi).

3. New Concepts

i) Windows 10 IoT Core operating system.

ii) Deployment of Windows lot on Raspberry Pi.

iii) Using Windows IoT Dashboard.

iv) Universal Windows Platform (UWP) applications.

4. Procedure

1 Go to the Windows 10 developer center.

2 Click Get Windows 10 IoT Core Dashboard to download the necessary application.

Input IMAGE(4)

3. Writing Windows IoT Core Image to SD Card

Input IMAGE(3)

4. Windows IoT Core Startup Screen

Input IMAGE(3)

5. Raspbrrry Pi Appearing in IoT Dashboard

Input IMAGE(5)

6. Raspberry Pi Device Portal in Browser

Input IMAGE(4)

5. Demo Application:

Display a simple message on the Raspberry Pi screen. (C# UWP):

```
using Windows.UI.Xaml;

using Windows.UI.Xaml.Controls;

namespace HelloloT

{

public sealed partial class MainPage: Page
```

```
{
public MainPage()
{
this.InitializeComponent();
TextBlock txt = new TextBlock();
txt.Text = "Hello Windows IoT Core!";
txt.FontSize = 40;
this.Content = txt;
        }
    }
}
```